

Practical-1	To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort
Date :	

1)Bubble sort

```
BEGIN
  INPUT n
  INPUT array A[0....n-1]

  FOR i ← 0 TO n-2 DO
    FOR j ← 0 TO n-i-2 DO
      IF A[j] > A[j+1] THEN
        TEMP ← A[j]
        A[j] ← A[j+1]
        A[j+1] ← TEMP
      ENDIF
    END FOR
  END FOR

  OUTPUT A

END
```

code:

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        // Flag to check if swapping happens
        bool swapped = false;

        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap adjacent elements
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
    }

    // If no swapping occurred, array is already sorted
    if (!swapped)
        break;
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    bubbleSort(arr, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output:

Output

Sorted array: 11 12 22 25 34 64 90

=== Code Execution Successful ===

Time complexity:

Time complexity :-

$$T(n) = (n-1) + (n-2) + (n-3) + \dots + 2 + 1$$

$$T(n) = \frac{n(n-1)}{2}$$
$$= \frac{n^2 - n}{2}$$

$$T(n) = n^2$$

$$T(n) = O(n^2)$$

Best case:- runs completely $\rightarrow O(n^2)$

Average case:- runs completely $\rightarrow O(n^2)$

Worst case:- still runs completely $\rightarrow O(n^2)$

2) Selection sort

```
BEGIN
INPUT n
INPUT array A[0.....n-1]

FOR i ← 0 TO n-2 DO
    min ← i
    FOR j ← i+1 TO n-1 DO
        IF A[j] < A[min] THEN
            min ← j
        ENDIF
    END FOR
    TEMP ← A[i]
    A[i] ← A[min]
    A[min] ← TEMP
END FOR

OUTPUT A
END
```

code:

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        // Find the smallest element
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        // Swap smallest element with current element
        swap(arr[i], arr[minIndex]);
    }
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    selectionSort(arr, n);

    cout << "Sorted array(Selection sort): ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output:

Output

```
Sorted array(selection sort): 11 12 22 25 64
```

```
=== Code Execution Successful ===
```

Time complexity:

Time complexity:-

Outer loop $\rightarrow n$ times $\rightarrow O(n)$

Inner Loop $\rightarrow T(n) = (n-1) + (n-2) + (n-3) + \dots + 2 + 1$

$$= \frac{n(n-1)}{2}$$

$$= \frac{n^2 - n}{2}$$

$$T(n) = O(n^2)$$

swap() $\rightarrow O(1)$

so the time complexity is $T(n) = O(n^2)$

3) Insertion sort

```
WHILE low ≤ high DO
    mid ← (low + high) / 2
    IF A[mid] = key THEN
        OUTPUT mid
        STOP
    ELSE IF A[mid] < key THEN
        low ← mid + 1
    ELSE
        high ← mid - 1
    END IF
END WHILE
OUTPUT "Element not found"
END
```

code:

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        // Move elements greater than key one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        // Insert key at the correct position
        arr[j + 1] = key;
    }
}

int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    insertionSort(arr, n);

    cout << "Sorted array(insertion sort): ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```


Output:

Output

Sorted array(insertion sort): 5 6 11 12 13

=== Code Execution Successful ===

Time complexity:

Time complexity:-

$$T(n) = (n-1) + (n-2) + (n-3) + \dots + 2 + 1$$
$$T(n) = \frac{n(n-1)}{2}$$
$$T(n) = O(n^2)$$

Best case:- Still searches even though it is sorted
A = 1 2 3 4 5 $O(n^2)$

Avg case:- Randomly arranged, still perform comparisons
A = 3 1 5 2 4 $O(n^2)$

Worst case:- Still performs same number of comparisons
A = 5 4 3 2 1

4) Merge sort

```
BEGIN
MergeSort(A, low, high)
  IF low < high THEN
    mid ← (low + high) / 2
    MergeSort(A, low, mid)
    MergeSort(A, mid + 1, high)
    Merge(A, low, mid, high)
  END IF
END
Merge(A, low, mid, high)
  Merge the two sorted subarrays into a temporary
  array copy the temporary array back into A
END
```

code:

```
#include <iostream>
using namespace std;

void merge(int arr[], int low, int mid, int high) {
    int i = low;
    int j = mid + 1;
    int k = 0;

    int temp[100];

    while (i <= mid && j <= high) {
        if (arr[i] <= arr[j]) {
            temp[k] = arr[i];
            i++;
        } else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= high) {
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (i = low, k = 0; i <= high; i++, k++) {
        arr[i] = temp[k];
    }
}
```

```
void mergeSort(int arr[], int low, int high) {  
    if (low < high) {  
        int mid = (low + high) / 2;  
  
        // Divide  
        mergeSort(arr, low, mid);  
        mergeSort(arr, mid + 1, high);  
  
        // Merge  
        merge(arr, low, mid, high);  
    }  
}
```

```
int main() {  
    int arr[] = {38, 27, 43, 3, 9, 82, 10};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    mergeSort(arr, 0, n - 1);  
  
    cout << "Sorted array(merge sort): ";  
  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

Output:

Output

```
Sorted array(merge sort): 3 9 10 27 38 43 82
```

```
=== Code Execution Successful ===
```

Time complexity:

Merge sort Time complexity :-

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$T(n) = 2[2T\left(\frac{n}{4}\right) + \frac{n}{2}] + n$$

$$T(n) = 4T\left(\frac{n}{4}\right) + 2n$$

$$T(n) = 8T\left(\frac{n}{8}\right) + 3n$$

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn$$

$$\frac{n}{2^k} = 1$$

$$2^k = n$$

$$k = \log_2 n$$

So, $T(n) = nT(1) + n \log_2 n$

$$T(n) = n + n \log n$$

$$T(n) = O(n \log n)$$

Best case:- $O(n \log n)$

Avg case:- $O(n \log n)$

Worst case:- $O(n \log n)$

→ It remains same for all cases it still divides in recursive

5) Quick sort

```
QuickSort(A, low, high)
IF low < high THEN
    Pivot ← Partition(A, low, high)
    QuickSort(A, low, pivot-1)
    QuickSort(A, pivot+1, high)
END IF
Partition(A, low, high)
pivot ← A[low]
i ← low
j ← high
WHILE i < j DO
    WHILE A[j] ≤ pivot DO
        i ← i+1
    END WHILE
    IF i < j THEN
        SWAP A[i] A[j]
    END IF
END WHILE
SWAP A[low], A[i]
RETURN j
END
```

code:

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);

    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);

        // Divide
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```



```
int main() {  
    int arr[] = {10, 7, 8, 9, 1, 5};  
    int n = sizeof(arr) / sizeof(arr[0]);  
  
    quickSort(arr, 0, n - 1);  
  
    cout << "Sorted array(Quick sort): ";  
  
    for (int i = 0; i < n; i++) {  
        cout << arr[i] << " ";  
    }  
  
    return 0;  
}
```

Output:

```
Output  
Sorted array(quick sort): 1 5 7 8 9 10  
=== Code Execution Successful ===
```

Time complexity:

Time Complexity:-

1) Best case:-

$$T(n) = n \log_2 n$$

$$T(n) = O(n \log n)$$

2) Avg case:-

$$T(n) = 2T(n/2) + n$$

$$T(n) = 2[2T(n/4) + n/2] + n$$

$$T(n) = 4T(n/4) + 2n$$

$$T(n) = 8T(n/8) + 3n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$\frac{n}{2^k} = 1$$

$$k = \log_2 n$$

$$T(n) = n \log_2 n$$

$$T(n) = O(n \log n)$$

3) worst case:- Instead of dividing into two equal parts, we get

$$T(n) = n + (n-1) + (n-2) + \dots + 1$$

$$T(n) = \frac{n(n+1)}{2}$$

$$= \frac{n^2 + n}{2}$$

$$T(n) = O(n^2)$$

Conclusion:

We implemented Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort and studied their working and time complexities. Merge Sort and Quick Sort are generally more efficient for large datasets compared to the other three algorithms.
